

Tema 5. Polimorfismo

1. Brevemente, ¿qué es el "**polimorfismo**" y para qué sirve en programación orientada a objetos? ¿qué es la "**sobreescritura**" de métodos?

Polimorfismo

Es la capacidad de tratar objetos de distintas clases de forma uniforme a través de una misma interfaz, haciendo que respondan de manera diferente al mismo mensaje (método).

Ejemplo: varias clases implementan saludar(), pero cada una lo hace a su manera.

¿Para qué sirve?

Permite escribir código más genérico y reutilizable

Facilita la extensibilidad

Reduce dependencias concretas

Sobreescritura (override)

Es cuando una subclase redefine un método heredado de la clase padre, cambiando su comportamiento.

Requisitos típicos:

Mismo nombre

Misma firma (parámetros)

2. ¿En qué consiste la "**ligadura dinámica**" o "**enlace tardío**"? ¿qué relación tiene con el polimorfismo? ¿hay que indicarlos explícitamente al programar o depende esto del lenguaje? Compara C++ y Java. Indicalo después también para Python.

¿Qué es?

Es el mecanismo por el cual la llamada a un método se resuelve en tiempo de ejecución, en función del tipo real del objeto (no del tipo de la referencia).

- ◆ Relación con el polimorfismo

Es lo que hace posible el polimorfismo:

```
Soldado s = new Zapador();  
s.saludar(); // Se decide en ejecución → Zapador.saludar()
```

- ◆ ¿Hay que indicarlo explícitamente?

Depende del lenguaje:

En Java

Es automático por defecto para métodos no estáticos

No hay que indicar nada

Se puede usar `@Override` (opcional pero recomendable)

C++

La ligadura dinámica no es automática.

Hay que usar `virtual`.

Permite polimorfismo en tiempo de ejecución.

Java

La ligadura dinámica es el comportamiento por defecto en métodos de instancia.

No hace falta indicar `virtual`.

Base del polimorfismo OO en Java.

Python

Todo funciona dinámicamente.

No hace falta declarar tipos ni `virtual`.

Usa polimorfismo y además duck typing.

3. Pon un ejemplo sencillo en Java, de un `Soldado` , con un método `saluda` , con dos subclases: `Zapador` y `Artillero` , donde `Zapador` sobrescribe el método `saludar` , sustituyendo por completo su comportamiento. Ilustra el funcionamiento del polimorfismo creando un array de `Soldados` de dos

tipos y luego recorriéndolo empleando referencias de tipo Soldado y llamando a saludar .

```
class Soldado {
    public void saludar() {
        System.out.println("Saludo general de soldado");
    }
}
class Zapador extends Soldado {
    @Override
    public void saludar() {
        System.out.println("Zapador listo para desactivar minas");
    }
}
class Artillero extends Soldado {
    @Override
    public void saludar() {
        System.out.println("Artillero preparado para disparar");
    }
}
```

Uso del polimorfismo:

```
public class Main {
    public static void main(String[] args) {

        Soldado[] ejercito = new Soldado[3];

        ejercito[0] = new Soldado();
        ejercito[1] = new Zapador();
        ejercito[2] = new Artillero();

        for (Soldado s : ejercito) {
            s.saludar(); // llamada polimórfica
        }
    }
}
```

◆ ¿Qué ocurre aquí?

Todas las referencias son de tipo Soldado

Pero el comportamiento depende del tipo real del objeto
Gracias a la ligadura dinámica, se ejecuta el método correcto

4. Si sobrescribo un método, ¿puedo invocar el método base para trabajar a partir de su resultado? Haz que zapador cambie ligeramente la forma de saludar, que salude de forma normal, tal cual hace el soldado base, pero que además añada un "ZAPADOR A SUS ORDENES" ¿qué palabra clave del lenguaje has usado para invocar al método de la clase base?

Sí, puedes invocar el método de la clase base al sobrescribirlo y así reutilizar su comportamiento, añadiendo o modificando solo una parte.

◆ ¿Cómo se hace en Java?

Se utiliza la palabra clave `super`, que permite acceder a la implementación de la clase padre.

Cuando una clase hija sobrescribe un método, normalmente sustituye completamente al del padre.

Pero a veces quieres:

conservar lo que hacía el padre y añadir algo extra-> Ahí es donde se usa `super`.

```
class Soldado {
    public void saludar() {
        System.out.println("Saludo general de soldado");
    }
}

class Zapador extends Soldado {
    @Override
    public void saludar() {
        super.saludar(); // llamada al método de la clase base
        System.out.println("ZAPADOR A SUS ORDENES");
    }
}
```

◆ ¿Qué ocurre aquí?

Cuando llamas a:

```
Soldado s = new Zapador();  
s.saludar();
```

La salida será:

Saludo general de soldado
ZAPADOR A SUS ORDENES

->Conclusión

Sí, puedes reutilizar el comportamiento del padre

Se usa la palabra clave super

Esto permite extender en lugar de reemplazar completamente el método

5. Al sobrescribir un método en Java, ¿qué restricciones existen sobre los tipos de los parámetros y el tipo de retorno? ¿Qué diferencia hay entre sobreescritura (*overriding*) y sobrecarga (*overloading*)? ¿Para qué sirve la anotación `@Override` y por qué es recomendable usarla siempre?

◆ ¿Qué restricciones existen sobre los tipos de los parámetros y el tipo de retorno?

Para que sea una sobreescritura válida:

El método debe tener el mismo nombre

Los parámetros deben ser exactamente iguales

El tipo de retorno debe ser:

igual

o compatible (covariante)

→ Puedes hacer el método más accesible, pero no menos:

protected → public 

public → protected 

Excepciones (checked)

→ No puedes declarar excepciones más generales que las del método original.

No se pueden sobrescribir métodos final, static o private

(los static se “ocultan”, no se sobrescriben realmente)

◆ Overriding vs Overloading

✔ Sobreescritura (overriding)

Ocurre entre clase padre e hija

Misma firma

Se decide en tiempo de ejecución (polimorfismo, ligadura dinámica)

```
class A {  
    void metodo() {}  
}  
  
class B extends A {  
    @Override  
    void metodo() {} // overriding-> lo redefine  
}
```

✔ Sobrecarga (overloading)

Ocurre en la misma clase (o jerarquía)

Mismo nombre, distintos parámetros

Se decide en tiempo de compilación

```
class Calculadora {  
  
    int sumar(int a, int b) {  
        return a + b;  
    }  
  
    double sumar(double a, double b) {  
        return a + b;  
    }  
}
```

◆ ¿Para qué sirve @Override?

Es una anotación que indica que un método está sobrescribiendo uno de la superclase.

◆ ¿Por qué es recomendable usarla siempre?

El compilador verifica que realmente estás sobrescribiendo

Evita errores sutiles (por ejemplo, equivocarte en un parámetro)

👉 Ejemplo de error sin @Override:

```

class A {
    void metodo(int x) {}
}

class B extends A {
    void metodo(double x) {} // ❌ No sobrescribe, sobrecarga sin querer
}

```

Con @Override, esto daría error de compilación.

6. Entonces, cuando se estudia Java, ¿se emplea el polimorfismo desde el principio? Por ejemplo, sobrescribiendo toString o sobrescribiendo equals , ¿ya estoy usando polimorfismo?

Sí: desde el principio ya estás usando polimorfismo en Java, aunque no siempre se explique explícitamente.

♦ ¿Por qué?

Porque todas las clases heredan de Object, que define métodos como:

```

toString()
equals()
hashCode()

```

Cuando tú los sobrescribes, estás participando en un comportamiento polimórfico.

♦ Ejemplo con toString()

```

class Persona {
    String nombre;

    @Override
    public String toString() {
        return "Persona: " + nombre;
    }
}

```

Y luego:

```
Object obj = new Persona();  
System.out.println(obj.toString());
```

Aunque la referencia es de tipo Object, se ejecuta el toString() de Persona.
Eso es polimorfismo + ligadura dinámica.

♦ ¿Y con equals()?

Exactamente lo mismo:

Estás redefiniendo un método de la superclase

El comportamiento depende del tipo real del objeto en ejecución

Por tanto, también es polimorfismo.

♦ Idea clave

- ✓ Sobrescribir métodos = usar polimorfismo
- ✓ Aunque no uses jerarquías complejas
- ✓ Aunque no declares explícitamente “polimorfismo”

Conclusión

Sí:

Cuando sobrescribes toString(), equals() u otros métodos heredados de Object, ya estás usando polimorfismo, incluso en los ejemplos más básicos de Java.

7. ¿Qué es una "clase abstracta"? ¿Qué es un "método abstracto"? ¿Puedo crear instancias de una clase abstracta? Pongamos un ejemplo en Java: Redefinamos soldado , hagamos que, además del método saluda que ya tenía, tenga un método atacar , que sea abstracto y que cada tipo de soldado haga su acción cuando se le pida atacar. ¿Donde debemos poner abstract ?

♦ ¿Qué es una clase abstracta?

Una clase abstracta en Java es una clase que:

-No se puede instanciar directamente

-Puede contener:

Métodos normales (con implementación)

Métodos abstractos (sin implementación)

Sirve como plantilla base para otras clases

->Se usa cuando tiene sentido definir una idea general, pero no completa.

♦ ¿Qué es un método abstracto?

Es un método que:

No tiene cuerpo (sin implementación)

Solo define su firma

Obliga a las subclases a implementarlo

```
abstract void atacar();
```

♦ ¿Puedo crear instancias de una clase abstracta?

No.

No puedes hacer:

```
Soldado s = new Soldado(); // ✗ ERROR
```

✓ Solo puedes instanciar subclases concretas.

```
public abstract class Soldado {  
  
    public void saludar() {  
        System.out.println("Saludo general de soldado");  
    }  
  
    public abstract void atacar(); // método abstracto  
}
```

👉 Aquí:

La clase Soldado es abstracta

atacar() no tiene implementación

✓ Subclases concretas

◆ Zapador

```
public class Zapador extends Soldado {  
  
    @Override  
    public void atacar() {  
        System.out.println("El zapador coloca explosivos");  
    }  
}
```

◆ Artillero

```
class Artillero extends Soldado {  
  
    @Override  
    public void atacar() {  
        System.out.println("El artillero dispara su cañón");  
    }  
}
```

✓ Uso polimórfico

```
public class Main {  
    public static void main(String[] args) {  
  
        Soldado[] ejercito = {  
            new Zapador(),  
            new Artillero()  
        };  
  
        for (Soldado s : ejercito) {  
            s.saludar();  
            s.atacar(); // comportamiento específico  
        }  
    }  
}
```

◆ ¿Dónde va abstract?

En Java:

✓ En la clase:

abstract class Soldado

✓ En el método:

public abstract void atacar();

8. ¿Qué efecto tiene la palabra clave `final` sobre métodos y clases en Java? ¿Cómo se relaciona con el polimorfismo? ¿Conoces algún ejemplo de clase `final` en la propia API estándar de Java?

◆ ¿Qué efecto tiene `final` en Java?

La palabra clave `final` restringe la modificación en distintos niveles.

◆ 1. Métodos `final`

Un método declarado como `final`:

No puede ser sobrescrito (no overriding) en subclases.

Se hereda, pero su comportamiento queda “fijo”.

```
class A {  
    public final void mostrar() {  
        System.out.println("Método final");  
    }  
}  
  
class B extends A {  
    // ✗ ERROR: no se puede sobrescribir  
    /*  
    public void mostrar() {  
    }  
    */  
}
```

Esto bloquea el polimorfismo por sobrescritura en ese método

◆ 2. Clases `final`

Una clase `final`:

No puede tener subclases

Es “hoja” en la jerarquía de herencia

```
final class MiClase {  
}  
// ✖ ERROR  
class Otra extends MiClase {  
}
```

- ◆ Relación con el polimorfismo

El polimorfismo en Java depende de la sobreescritura de métodos:

Si un método es final → no hay overriding → no hay polimorfismo en ese método

Si una clase es final → no hay subclases → no hay polimorfismo por herencia

En resumen:

final reduce o elimina el polimorfismo estructural

- ◆ ¿Por qué se usa final?

Seguridad (evitar modificaciones no deseadas)

Control del diseño

Optimización potencial del compilador/JVM

Evitar jerarquías peligrosas o mal diseñadas

- ◆ Ejemplo en la API estándar de Java

Sí, hay varias clases final en la API de Java.

- ✓ Ejemplo claro:

String

Es una clase final

No puede ser heredada

```
String s = "hola";
```

- ◆ ¿Por qué String es final?

Garantiza inmutabilidad

Evita que alguien cambie su comportamiento

Permite optimizaciones internas (como el string pool)

RESUMEN

final en clases: prohíbe heredar

final en métodos: prohíbe sobreescribir

9. En Java, qué son las "interfaces"? ¿Son como clases abstractas? ¿Una clase puede implementar más de una interfaz?

◆ ¿Qué son las interfaces en Java?

Una interfaz es un tipo especial que define un contrato:
indica qué se debe hacer, pero no cómo se hace.

Es decir:

Solo declara métodos (normalmente)

Las clases que la implementan deben proporcionar la implementación

◆ ¿Son como clases abstractas?

Se parecen, pero no son lo mismo.

✓ Similitudes:

Ambas pueden definir métodos sin implementación

Ambas se usan para polimorfismo

No siempre se instancian directamente

✗ Diferencias clave:

Clases abstractas	Interfaces
Pueden tener estado (atributos)	No tienen estado (solo constantes)
Pueden tener métodos con implementación	Tradicionalmente no (aunque Java moderno permite default)
Herencia simple	Permiten “herencia múltiple de tipo”

```
public abstract class Animal {  
    void dormir() {  
        System.out.println("Durmiendo");  
    }  
  
    abstract void hacerSonido();  
}
```

```
public interface Volador {  
    void volar();  
}
```

- ◆ ¿Puede una clase implementar varias interfaces?
- ✔ Sí, en Java puede implementar múltiples interfaces

Esto es clave porque Java no permite herencia múltiple de clases.

```
interface Volador {  
    void volar();  
}
```

```
interface Nadador {  
    void nadar();  
}
```

```
class Pato implements Volador, Nadador {  
  
    public void volar() {  
        System.out.println("El pato vuela");  
    }  
  
    public void nadar() {  
        System.out.println("El pato nada");  
    }  
}
```

- ◆ ¿Por qué esto es importante?

Las interfaces permiten:

- ✔ Diseñar sistemas más flexibles
- ✔ Simular herencia múltiple de comportamiento
- ✔ Desacoplar código (muy usado en frameworks)

10. Vamos a poner un ejemplo nuevo con polimorfismo. Queremos implementar una clase Punto , con un método calcularDistanciaA , que

permite calcular la distancia a otro `Punto` . Sin embargo, como queremos trabajar con puntos 2D y 3D, haz que ese método sea abstracto y haya dos implementaciones de ese cálculo de distancia. Emplea `instanceof` y *downcasting* para verificar que se recibe un punto compatible y poder calcular correctamente la distancia siempre entre puntos del mismo subtipo. Aprovecha este diseño para crear ahora una clase `Linea` , que acepta `Punto` , sin saber de qué tipo es, y es capaz de dar su longitud independientemente de las dimensiones de sus puntos (las cuales desconoce).

```
public class Punto2D extends Punto {

    private double x, y;

    public Punto2D(double x, double y) {
        this.x = x;
        this.y = y;
    }

    @Override
    public double calcularDistanciaA(Punto otro) {

        if (otro instanceof Punto2D otro2D) {

            double dx = this.x - otro2D.x;
            double dy = this.y - otro2D.y;

            return Math.sqrt(dx * dx + dy * dy);
        }

        throw new IllegalArgumentException("El punto no es 2D");
    }
}
```

```

public class Punto3D extends Punto {

    private double x, y, z;

    public Punto3D(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    @Override
    public double calcularDistanciaA(Punto otro) {

        if (otro instanceof Punto3D otro3D) {

            double dx = this.x - otro3D.x;
            double dy = this.y - otro3D.y;
            double dz = this.z - otro3D.z;

            return Math.sqrt(dx * dx + dy * dy + dz * dz);
        }

        throw new IllegalArgumentException("El punto no es 3D");
    }
}

```

```

public class Linea {

    private Punto inicio;
    private Punto fin;

    public Linea(Punto inicio, Punto fin) {
        this.inicio = inicio;
        this.fin = fin;
    }

    public double longitud() {
        return inicio.calcularDistanciaA(fin);
    }
}

```

```

public class Main {

```



```

public static void main(String[] args) {

    Punto p1 = new Punto2D(0, 0);
    Punto p2 = new Punto2D(3, 4);

    Linea l1 = new Linea(p1, p2);

    System.out.println(l1.longitud()); // 5.0

    Punto p3 = new Punto3D(0, 0, 0);
    Punto p4 = new Punto3D(1, 2, 2);

    Linea l2 = new Linea(p3, p4);

    System.out.println(l2.longitud()); // 3.0
}
}

```

11. ¿Qué es la "herencia de interfaces" en Java? ¿Existe "herencia múltiple de interfaces"? Pon un ejemplo de una interfaz `Fichero` que tenga un método para leer su contenido en forma de `String` y luego dicha interfaz sea extendida por otra que sea `FicheroEscribible` que permita enviar contenido e incluso eliminar el fichero.

La herencia de interfaces en Java consiste en que una interfaz puede extender otra interfaz, heredando sus métodos (sin implementación, salvo default o static). Esto permite construir jerarquías más específicas a partir de comportamientos más generales.

¿Existe herencia múltiple de interfaces?

Sí.

En Java, una interfaz puede extender varias interfaces a la vez, algo que no se permite con clases.

Ejemplo paso a paso

Interfaz base: `Fichero`

Define un comportamiento básico: leer contenido.

```
public interface Fichero {  
    String leer();  
}
```

Interfaz derivada: FicheroEscribible

Extiende Fichero y añade más funcionalidades.

```
public interface FicheroEscribible extends Fichero {  
    void escribir(String contenido);  
    void eliminar();  
}
```

Aquí FicheroEscribible hereda el método leer() de Fichero y añade nuevos métodos.

*Ejemplos con herencia múltiple de interfaces

```
public interface Borrable {  
    void eliminar();  
}  
  
public interface Editable extends Fichero, Borrable {  
    void escribir(String contenido);  
}
```

Editable hereda:

leer() de Fichero

eliminar() de Borrable

y añade escribir()